

Generalized Module (Dependency?) Mapper

Document: [P1842R0](#)
Audience: SG15
Authors: Boris Kolpackov (Code Synthesis / build2)
Reply-To: boris@codesynthesis.com
Date: 2019-08-04

Abstract

This paper suggests generalizing the module mapper protocol described in [P1184](#) to also handle headers as well as potential future translation unit dependencies such as `std::embed` ([P1040](#)).

Contents

- 1 [Background](#)
- 2 [Communication](#)
- 3 [Protocol](#)
 - 3.1 [Dynamic Mapper](#)
 - 3.1.1 `IMPORT`
 - 3.1.2 `INCLUDE`
 - 3.2 [Static Mapper](#)
- 4 [Questions and Answers](#)

4.1 [Is there implementation experience?](#)

4.2 [Is there usage experience?](#)

5 [Acknowledgments](#)

1 Background

Because header units affects the preprocessor, they introduce a significant complication to the dependency graph discovery (refer to [P1184](#) for details). A dynamic module mapper is currently the only approach that we know of that allows dealing with this complication in the general case (that is, without relying on a pre-compilation step or manual dependency specification) and without requiring an additional mechanism in the compiler (such as the ability to preprocess with isolation textual headers in lieu of loading BMIs). As a result, in `build2`, we have decided to use the module mapper approach to handle header units and include translation.

Our initial attempt used GCC's module mapper to discover and handle header unit importation and the `-M` option family for header dependency discovery. However, it quickly became clear that there is a significant overlap between the two mechanisms. In fact, because of the include translation, the mapper gets notified about most headers reported by `-M`: the only exceptions are the predefined (forced) and command line (`-include`) headers.

More importantly, the mapper approach seemed like a promising way to resolve many long-standing issues with handling auto-generated headers. To give some background, in the `-M` option family, auto-generated headers are normally handled using `-MG` which instructs the compiler to not fail on encountering non-existent headers. The build system then detects such headers in the `-M` output, generates them, and re-executes the compiler.

However, this approach, besides being inefficient, also has many issues and corner cases (listed in the order of increased difficulty to deal with):

- Outdated header: The build system has to detect when an auto-generated header exists but is out of date, update it, and, again, re-execute the compiler.
- Wrong header: If the auto-generated header does not exist, the compiler may find and include an identically-named but unrelated header that is found in one

of the further `-I` directories.

- Outdated/wrong header causes an error: Including an outdated or wrong header may trigger a fatal preprocessor error (e.g., via an `#error` directive) that would disappear if only the header could be regenerated.

In contrast, the mapper approach would have the ability to sidestep all these issues because it would give the build system a chance to act *before* preprocessing a header.

Finally, the mapper can also be easily extended to handle potential future dependencies of translation units, such as those in the `std::embed` proposal ([P1040](#)).

The following sections describe the generalized module mapper (now more accurately called *dependency mapper*) that we have [implemented in GCC](#) and then used in `build2` with good results.

2 Communication

GCC currently supports several module mapper communication media:

1. File.
2. Pipe (including compiler's `stdin / stdout`).
3. Program to spawn and then communicate via its `stdin / stdout`.
4. Socket/port to connect to (UNIX, IP).

The last two communication media may understandably raise security concerns. We, however, believe they can be omitted or made optional by an implementation if non-intrusive support for legacy build systems is not a priority.

To elaborate, in our experience, the most natural way to integrate the module mapper functionality into a build system is using the first two media (file and/or pipe). The build system spawns the compiler process and using a pipe is the most straightforward and efficient way of establishing bi-directional communication. Only when the build system cannot be easily modified, might other communication media be necessary.

3 Protocol

For the remainder of the paper we refer to the file-based mapper as *static* and the rest – as *dynamic*. The dynamic mapper uses the line-based request-response protocol. The static mapper, due to its nature, has a separate, more limited protocol. Refer to [P1184](#) for the protocol basics and to the following sections for the generalizations.

Theoretically, a static mapper can be implemented via something other than a file. For example, the compiler may read the static mapping from its `stdin`.

An implementation can reasonably be expected to support multiple static mappers and a single dynamic mapper for the same compilation.

One notable protocol feature described in [P1184](#) is request batching in the dynamic. However, with the relaxation of the preamble rules around macro importation, the compiler's ability to request multiple mappings in parallel is now limited to contiguous non-header unit imports. It is therefore unclear whether the extra complexity (both in the compiler and in the build system) justifies the now limited benefit. As a result, we propose that if implemented, this feature be made optional and its use negotiated via the *impl-extra* field in the `HELLO` request/response (see below).

3.1 Dynamic Mapper

The generalized protocol uses quoting to distinguish between modules and headers. The `""` and `<>` quoting are used for the corresponding styles of `include` and `import` directives while `' '` is used for the predefined (forced) and command line inclusion as well as in the contexts where translation or re-search is not allowed (in other words, `' '` implies final/immutable inclusion/importation).

Protocol synopsis (leading `>` marks a request from the compiler to the mapper and `<` – a response).

```

> HELLO ver kind ident [impl-extra...]

  < HELLO ver kind ident [impl-extra...]
  < ERROR msg

> EXPORT mod-name
> EXPORT 'hdr-name'

  < EXPORT bmi
  < ERROR msg

> DONE mod-name
> DONE 'hdr-name'

> IMPORT mod-name
> IMPORT <hdr-name> [hdr-path]
> IMPORT "hdr-name" [hdr-path]
> IMPORT 'hdr-name' hdr-path

  < SEARCH
  < IMPORT [bmi]
  < ERROR msg

> INCLUDE <hdr-name> [hdr-path]
> INCLUDE "hdr-name" [hdr-path]
> INCLUDE 'hdr-name' hdr-path

  < SEARCH
  < INCLUDE
  < IMPORT [bmi]
  < ERROR msg

```

Example exchange translating `<stdio.h>` inclusion to an import :

```
> HELLO 0 GCC main.cxx
< HELLO 0 build2 .
> INCLUDE 'stdc-predef.h' /usr/include/stdc-predef.h
< INCLUDE
> INCLUDE <stdio.h> /usr/include/stdio.h
< IMPORT
> IMPORT '/usr/include/stdio.h'
< IMPORT stdio.gcm
```

Example exchange importing an auto-generated header:

```
> HELLO 0 GCC main.cxx
< HELLO 0 build2 .
> INCLUDE 'stdc-predef.h' /usr/include/stdc-predef.h
< INCLUDE
> IMPORT <foo/data.h>
< SEARCH
< IMPORT <foo/data.h> libfoo/foo/data.h
< IMPORT libfoo/foo/data.gcm
```

3.1.1 **IMPORT**

```
> IMPORT mod-name
> IMPORT <hdr-name> [hdr-path]
> IMPORT "hdr-name" [hdr-path]
> IMPORT 'hdr-name' hdr-path

< SEARCH
< IMPORT [bmi]
< ERROR msg
```

The first form of the `IMPORT` request is made when importing a module or a module partition. Valid responses are `IMPORT` and `ERROR`.

The next two forms are used for importing header units that were imported using `<>` and `""` importation styles, respectively. If the compiler was able to resolve this header name to the header path, then this path is included into the request as *hdr-path*. Otherwise, *hdr-path* is absent. Valid responses for these two forms are `SEARCH`, `IMPORT`, and `ERROR`. The `SEARCH` response causes the compiler to re-search the header name and re-issue the `IMPORT` request with the (presumably) new header path.

Instead of requesting the compiler to re-search the header, the response could have included the desired header path directly. The difficult part about supporting something like this would be the need to reverse-map the returned path to an include directory so that mechanisms such as `include_next`, system header status, etc., all work correctly. And it seems the only way to do this reliably would be to search for files in the include directories and see if one of them matches the returned path in the same heavy-handed way as `#pragma once` (comparing file contents, etc).

If the header is not found (*hdr-path* is absent), then the `IMPORT` response should cause the compiler to issue the usual "header not found" diagnostics. In this case the *bmi* field is ignored and can be omitted.

The last form is used to import header units that cannot be re-searched. For example, this form of the `IMPORT` request is issued for include directives that have been translated to import (see below).

3.1.2 `INCLUDE`

```
> INCLUDE <hdr-name> [hdr-path]
> INCLUDE "hdr-name" [hdr-path]
> INCLUDE 'hdr-name' hdr-path
```

```
< SEARCH
< INCLUDE
```

```
< IMPORT [bmi]  
< ERROR msg
```

The first two forms of the `INCLUDE` request are analogous to the corresponding `IMPORT` forms. The `INCLUDE` response signals that the header should be textually included while the `IMPORT` response signals that it should be translated to an import. The `IMPORT` response may optionally specify the BMI. If the BMI is omitted then the compiler should issue a separate `IMPORT` request.

Replying with just `IMPORT` could be useful if, for example, the mapping is split between dynamic and static mappers.

Similar to `IMPORT`, if the header is not found (*hdr-path* is absent), then the `INCLUDE` or `IMPORT` response should cause the compiler to issue the usual "header not found" diagnostics. In this case the *bmi* field in the `IMPORT` response is ignored and can be omitted.

The last form is used to include headers that can neither be re-searched nor translated.

3.2 Static Mapper

The static mapper specifies one module or header to BMI mapping per line in the following form:

```
[prefix] mod-name bmi  
[prefix] 'hdr-path' bmi  
[prefix] !'hdr-path' [bmi]
```

Note that the same format is used both to provide the input mapping for imported modules/headers as well as the output mapping for writing a module/header BMI.

A line prefix may be specified in an implementation-defined manner (for example, as part of the command line option that specifies the mapper file). If specified, then

only lines that begin with such a prefix are considered (the prefix itself is ignored). Leading (after the line prefix, if any) and trailing whitespaces as well as blank lines are ignored.

Specifying the line prefix is supported by GCC but this functionality is not described in [P1184](#).

The line prefix allows reusing existing files, such as the venerable `.d` file, for storing the module mapping information.

The last form (with the leading `!`) is used to signal that including this header should be translated to an import. In this form specifying the BMI is optional.

It may be desirable to allow separating the specification of header to BMI mapping and include translation, for example, in different mapper files. At the same time we expect it to be common for these specifications to be combined.

4 Questions and Answers

4.1 Is there implementation experience?

Yes, an implementation is available in the [boris/c++-modules-ex](#) GCC branch.

4.2 Is there usage experience?

Yes, the [build2](#) [build system](#) implements support for modules and header units (including include translation) in GCC using this generalized mapper.

5 Acknowledgments

This work is based on Nathan Sidwell's [P1184](#) and module mapper implementation in GCC. The module mapper idea was originally conceived (according to [P1184](#)) in a discussion between Nathan Sidwell, Richard Smith, and David Blaikie.